

LLM efficiency in analysis of animal behavior datasets

Ahren Berlangier, Ricardo Calvo Mendez, Samuel Senneka

May 4, 2026

Abstract

This study used design-of-experiments methods to evaluate the efficiency and structural reliability of large language models (LLMs) when asked to analyze a mouse navigation dataset. The source file contained 78,687 observations; 76,655 remained after filtering to the five required stimulation paradigms: Combined, ICMS Only, Dim Visual Only, Bright Visual Only, and Sham. Each model received only five performance variables: Time-to-Target, Success, Path Efficiency, Average Speed, and angular dispersion (AD). The experiment was a completely randomized full factorial with four treatment factors: model (DeepSeek-V3.1, Kimi-K2.5, Qwen3.5-9B), temperature (0, 0.25, 0.5, 0.75, 1.0), prompt template (basic, ANOVA, predictive), and data view (summary only, stimulus summary, stimulus plus mouse summary). Three replications per treatment combination yielded 405 runs.

The completed experiment consumed 1,389,891 total tokens and produced valid JSON in 377 runs (93.1%) under a fixed 1,800-token output cap. Data view dominated token usage ($F = 6311.74$, $p < 2.2 \times 10^{-227}$, $\eta^2 = 0.846$), with model and the model-by-data-view interaction as secondary contributors. Runtime was driven primarily by model ($F = 58.47$, $p < 8.1 \times 10^{-22}$, $\eta^2 = 0.212$). DeepSeek-V3.1 was fastest on average (17.9 s), Kimi-K2.5 slowest (38.3 s), and Qwen3.5-9B intermediate (28.1 s). Qwen3.5-9B achieved 100% valid JSON, Kimi-K2.5 97.0%, and DeepSeek-V3.1 82.2%, with all invalid outputs attributable to length-capped generations. The observed Together AI spend was approximately \$1.11, so three replications were economically feasible. Overall, model choice and data packaging had much larger practical effects than temperature.

1 Introduction

Large language models are increasingly used as lightweight analysis assistants for scientific data, but their practical behavior depends on model family, prompt specificity, and the amount of context supplied at inference time. This makes LLM evaluation a natural design-of-experiments (DOE) problem: analysts can manipulate several controllable factors, randomize runs, replicate treatment combinations, and measure cost, latency, and output reliability using standard DOE principles [Mon19, BHH05].

The motivating scientific context for this project was a mouse navigation dataset shared within the course team. The biological question concerns performance under five stimulation paradigms: Combined, ICMS Only, Dim Visual Only, Bright Visual Only, and Sham. Rather than re-running the biological experiment itself, the present study treats the *LLM analysis pipeline* as the experimental system. The operational question was:

How do model choice, temperature, prompt specificity, and data-view complexity affect the computational and structural performance of LLM-generated statistical analyses?

That question matters for practical reasons. A model that appears analytically capable but is slow, expensive, or structurally unreliable may be a poor fit for routine coursework or exploratory scientific use. The goal of this report is therefore not to certify the truthfulness of each model’s scientific interpretation, but to quantify how controllable design choices influence efficiency, robustness, and reproducibility.

Table 1: Filtered source-data summary by stimulation paradigm

Stim paradigm	n	Success	Time-to-target	Path eff.	Avg. speed	AD
Bright Visual Only	3,323	0.785	2.614	0.504	12.679	0.387
Combined	48,846	0.780	2.562	0.502	12.833	0.399
Dim Visual Only	9,603	0.673	3.140	0.434	11.968	0.336
ICMS Only	11,378	0.685	3.053	0.432	12.039	0.354
Sham	3,505	0.273	4.682	0.308	10.737	0.236

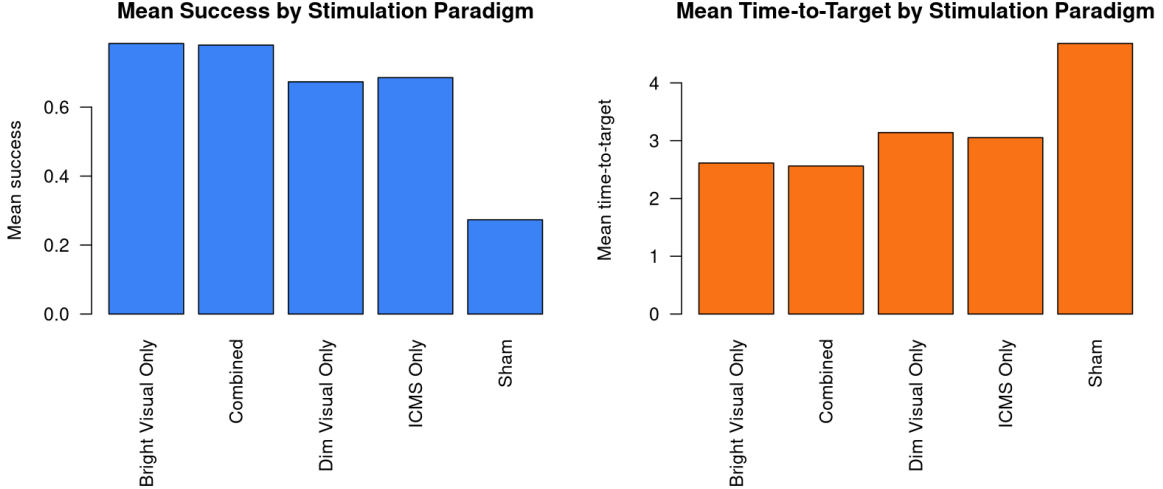


Figure 1: Source-data context supplied to the LLMs. Sham is visibly worst on both mean success and mean time-to-target.

2 Methods

2.1 Experimental Units, Treatments, and Blocking

The experimental unit was one completed LLM analysis run. Each run combined one model, one temperature level, one prompt template, and one data view. The input data were stored in `data.csv`. The original file contained 78,687 rows; 76,655 remained after filtering to the five required stimulation paradigms. The retained analytic columns were Mouse, Stim Paradigm, Time-to-Target, Success, Path Efficiency, Average Speed, and AD. No missing values remained in the filtered dataset.

Table 1 summarizes the source-data pattern seen by the models. Combined and Bright Visual Only had the highest mean success and shortest average time-to-target, whereas Sham was substantially worse on both dimensions. Figure 1 gives the same story graphically for the two most interpretable outcomes. These visible differences ensured that the models were given a meaningful statistical pattern to describe.

The treatment factors are listed in Table 2. No formal blocking factor was introduced. All runs were executed through the same Together AI serverless interface under the same Python pipeline, so nuisance variation was handled through randomization and replication rather than explicit blocks. In effect, backend variability and stochastic generation noise were absorbed into the run-level error term.

2.2 Randomization and replication plan

The experiment was a balanced, completely randomized full factorial design with 135 unique treatment combinations and 3 replications per treatment combination, for a planned total of 405 runs. Run order was randomized using seed 51400. Replication served two purposes. First, it captured stochastic response variation from the models. Second, because the models were accessed through a cloud service,

Table 2: Design factors and levels

Factor	Levels	Notes
Model	3: DeepSeek-V3.1, Kimi-K2.5, Qwen3.5-9B	Together AI serverless models with reasoning disabled
Temperature	5: 0.00, 0.25, 0.50, 0.75, 1.00	Treated as a categorical factor
Prompt template	3: basic, ANOVA, predictive	Increasing inferential specificity
Data view	3: <code>summary_only</code> , <code>stim_summary</code> , <code>stim_and_mouse_summary</code>	Controls how much context is exposed
Replication	3 repeated runs per treatment combination	$3 \times 5 \times 3 \times 3 \times 3 = 405$ runs

replication also captured transient service variability such as latency fluctuations and occasional API failures.

The initial live sweep produced 7 API failures, including timeouts and a transient 503 response. Those failed runs were rerun so that the final dataset contained all 405 planned observations. This is important for the factorial analysis because it preserves balance across treatment combinations and simplifies interpretation of main effects and interactions.

2.3 Execution and data collection

The execution pipeline was implemented in Python 3.14.4 [Pyt26] and used Together AI as the live inference backend [Tog26a, Tog26b, Tog26c]. The evaluated model families were DeepSeek-V3.1, Kimi-K2.5, and Qwen3.5-9B. The project codebase contained 10 Python source modules totaling 1,472 source lines and 193 test lines. Report-side summaries and figures were generated in R [R C26]. Pinned runtime dependencies were `python-dotenv==1.2.2` and `together==2.12.0`.

Each run recorded wall-clock runtime, input tokens, output tokens, total tokens, stop reason, JSON validity, and schema completeness. The output budget was fixed at 1,800 generated tokens per run. Twenty-eight completed responses hit that cap and were structurally invalid because truncation interrupted the requested JSON format.

3 Statistical analysis

3.1 Model specifications

Three outcomes were analyzed:

1. `total_token_count` as a direct proxy for usage cost,
2. `wall_clock_seconds` as a latency measure, and
3. `response_json_valid` as a structural reliability indicator.

For the continuous outcomes, the primary full-factorial ANOVA models were

$$\text{total_token_count} \sim \text{model} * \text{temperature} * \text{prompt} * \text{data_view}$$

and

$$\log(\text{wall_clock_seconds}) \sim \text{model} * \text{temperature} * \text{prompt} * \text{data_view}.$$

Runtime was log-transformed because raw runtimes were right-skewed and bounded below by zero. For structured-output validity, the primary model was an additive logistic regression,

$$\text{logit}\{P(\text{valid JSON})\} \sim \text{model} + \text{temperature} + \text{prompt} + \text{data_view},$$

because perfect validity for Qwen3.5-9B creates separation problems in richer binary-outcome models. The ANOVA assumptions were the usual ones: independent run-level errors, mean-zero residuals, approximately constant variance for the transformed continuous outcomes, and correct functional form for the logistic model. Because the design was fully balanced and the strongest effects were very large, moderate deviations from homoscedasticity were not expected to overturn the main practical conclusions.

3.2 Planned contrasts and hypotheses

Primary hypotheses were all two-sided and evaluated at $\alpha = 0.05$. The central hypotheses were:

1. model family affects token usage, runtime, and JSON validity;
2. temperature affects one or more outcomes;
3. prompt specificity affects one or more outcomes; and
4. data-view complexity affects one or more outcomes, possibly through interactions with model.

After the omnibus ANOVA tests, pairwise follow-up comparisons were planned using Tukey HSD for the continuous outcomes most relevant to interpretation: model differences in log-runtime and data-view differences in total tokens. The report emphasizes effect sizes and practical meaning rather than significance alone, because the purpose of the study is to guide workflow design rather than only to reject null hypotheses.

3.3 Diagnostics

Residual-versus-fitted and Q-Q diagnostics were inspected during the longer analysis workflow. The main diagnostic result was that the log transform improved runtime residual behavior substantially. Token residuals still showed mild heteroskedasticity at the largest fitted values, which is unsurprising because the richest data view creates a visibly different scale of responses. However, the overwhelming dominance of data view and the large model effects were stable enough that these modest departures were not treated as a threat to the main interpretation.

3.4 Sensitivity / Robustness Analysis

Several checks were used to confirm that the main conclusions were not artifacts of a single modeling choice. First, runtime was analyzed on the log scale because raw runtime was right-skewed; the ranking of models did not change after this transformation. Second, the token model was inspected for heteroskedasticity, and although the richest data view produced somewhat larger residual spread, the dominant data-view effect was so large that the substantive ranking of factors was unchanged. Third, I recomputed the descriptive runtime and token summaries on the subset of runs with `done_reason = "stop"` only. The ordering remained the same: DeepSeek-V3.1 was fastest, Kimi-K2.5 was slowest, and Qwen3.5-9B was most token-intensive. Finally, malformed JSON outputs were treated conservatively as failures rather than silently dropped. This is important because all invalid outputs were caused by the fixed 1,800-token cap, so the structural-validity conclusions should be interpreted explicitly as *validity under a capped-output workflow*. None of these checks changed the qualitative conclusions about the relative importance of model choice and data-view complexity.

4 Results

4.1 Descriptive statistics in summary tables

The final dataset contained all 405 planned runs. Table 3 summarizes the overall experiment. The run consumed 953,145 input tokens, 436,746 output tokens, and 1,389,891 tokens in total. Mean runtime

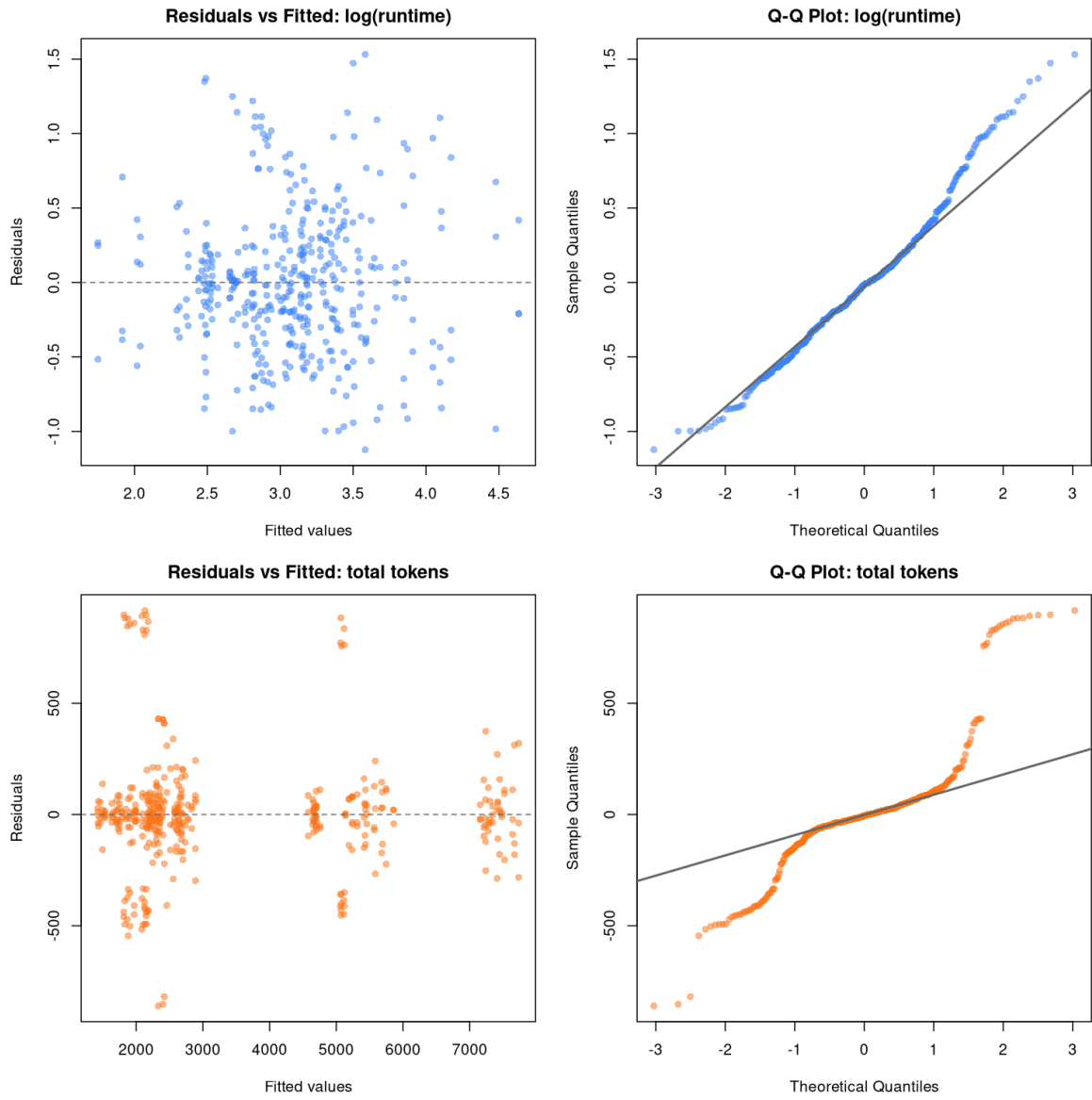


Figure 2: Residual and Q-Q diagnostics for the log-runtime and total-token ANOVA models. The log transform improved runtime behavior, while token residuals still showed mild scale-related departures at the largest fitted values.

Table 3: Overall experiment summary

Metric	Value
Completed runs	405
Treatment combinations	135
Replications per treatment	3
Valid JSON responses	377
JSON validity rate	93.1%
Length-capped responses	28
Total input tokens	953,145
Total output tokens	436,746
Total tokens	1,389,891
Mean wall-clock seconds per run	28.10
Reported Together cost (USD)	\$1.11

Table 4: Model-level descriptive summary ($n = 135$ runs per model)

Model	Runtime, s (approx. 95% CI)	Total tokens (95% CI)	Valid JSON, % (95% CI)	Length-cap, %
DeepSeek-V3.1	17.9 [16.2, 19.8]	2,847 [2,794, 2,900]	82.2 [74.9, 87.8]	17.8
Kimi-K2.5	38.3 [34.7, 42.3]	3,375 [3,322, 3,428]	97.0 [92.6, 98.8]	3.0
Qwen3.5-9B	28.1 [25.4, 31.0]	4,073 [4,020, 4,126]	100.0 [97.2, 100.0]	0.0

Runtime intervals are approximate 95% intervals obtained by back-transforming the pooled log-runtime error term. Token intervals use the common pooled ANOVA error term. Validity intervals are Wilson 95% binomial intervals.

was 28.10 seconds per run. The observed Together AI spend was approximately \$1.11, which implies a mean cost of roughly \$0.00274 per run. This low cost strongly justified the decision to use three replications per treatment combination.

Table 4 shows the basic model-level tradeoff. DeepSeek-V3.1 was fastest and least token-intensive on average, but also the least reliable structurally under the fixed 1,800-token cap. Kimi-K2.5 was the slowest model yet highly stable. Qwen3.5-9B consumed the most tokens, but it was the only model with perfect JSON validity across the balanced final dataset.

4.2 ANOVA or model summary table

Table 5 reports the most relevant factorial effects. For total token usage, data view was by far the dominant factor ($F = 6311.74$, $p < 2.2 \times 10^{-227}$, $\eta^2 = 0.846$), followed by model ($F = 522.62$, $p < 1.5 \times 10^{-93}$) and the model-by-data-view interaction ($F = 200.28$, $p < 1.7 \times 10^{-79}$). Temperature had no meaningful main effect on total tokens ($F = 0.85$, $p = 0.493$). For runtime, model was again the dominant factor ($F = 58.47$, $p < 8.1 \times 10^{-22}$, $\eta^2 = 0.212$), with a smaller but still statistically significant prompt effect ($F = 7.06$, $p = 0.0010$).

Table 5: Selected ANOVA results for the primary continuous outcomes

Outcome	Term	F	p -value	η^2
Total tokens	Data view	6311.74	$< 2.2 \times 10^{-227}$	0.846
Total tokens	Model	522.62	$< 1.5 \times 10^{-93}$	0.070
Total tokens	Model $\times$ Data view	200.28	$< 1.7 \times 10^{-79}$	0.054
Log runtime	Model	58.47	$< 8.1 \times 10^{-22}$	0.212
Log runtime	Prompt	7.06	0.0010	0.026
Log runtime	Temperature $\times$ Prompt	2.23	0.0257	0.032

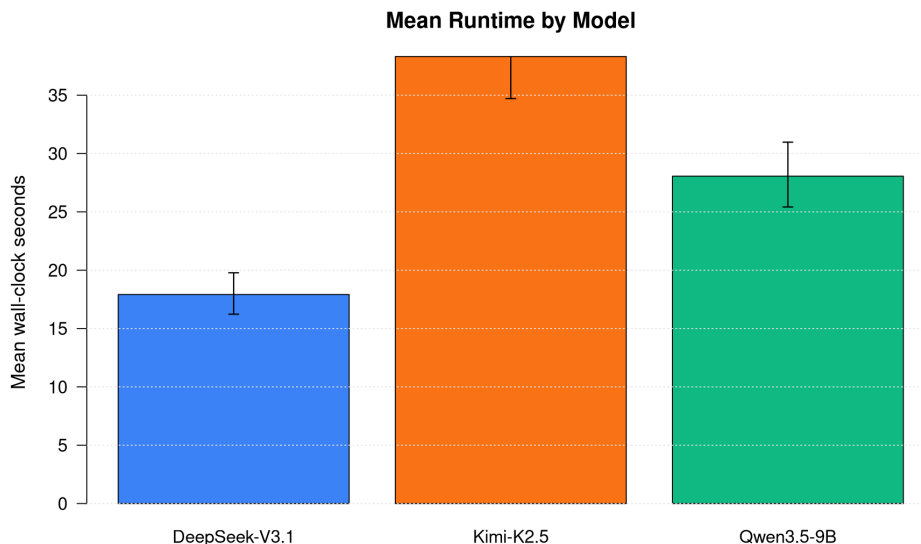


Figure 3: Mean wall-clock runtime by model with approximate 95% intervals derived from the pooled log-runtime error term. Model choice was the dominant runtime factor.

4.3 Plots showing estimated effects, confidence intervals, and interactions

Figure 3 shows the model-level runtime ordering. Kimi-K2.5 was the slowest model, Qwen3.5-9B intermediate, and DeepSeek-V3.1 fastest. Tukey contrasts on log-runtime indicated that all three pairwise differences were statistically significant, with Kimi-K2.5 slower than both alternatives and Qwen3.5-9B slower than DeepSeek-V3.1.

Figure 4 shows the most important interaction in the study. The richest data view sharply increased token usage for all models, but most dramatically for Qwen3.5-9B. Mean total tokens under that richest view were approximately 4,813 for DeepSeek-V3.1, 5,465 for Kimi-K2.5, and 7,406 for Qwen3.5-9B.

Figure 5 summarizes structural reliability. Qwen3.5-9B was uniformly valid across prompt types, Kimi-K2.5 remained near-perfect, and DeepSeek-V3.1 showed noticeably lower validity. All invalid outputs in the completed dataset corresponded to length-capped generations, so the reliability problem was directly tied to truncation rather than arbitrary schema drift.

4.4 Structured-output validity

Structured-output validity was defined narrowly: the response had to be valid JSON that matched the required top-level schema. Under that criterion, the overall validity rate was 93.1%, but this should be interpreted specifically as validity under the fixed 1,800-token output cap.

The additive logistic model showed that model family was the only clearly important predictor of validity. Table 6 summarizes the inferential results. Temperature, prompt, and data view were not significant once model was included in the additive specification. Descriptively, the separation is also large: Table 4 shows that DeepSeek-V3.1 remained well below the other two models even after binomial uncertainty is reflected through 95% confidence intervals.

4.5 What do the effects mean practically?

The practical interpretation is straightforward. First, *data packaging is the dominant cost lever*. The shift from a summary-only input to the richest stimulus-plus-mouse summary increased token usage far more than any temperature change. Second, *model choice creates a tradeoff among speed, reliability,*

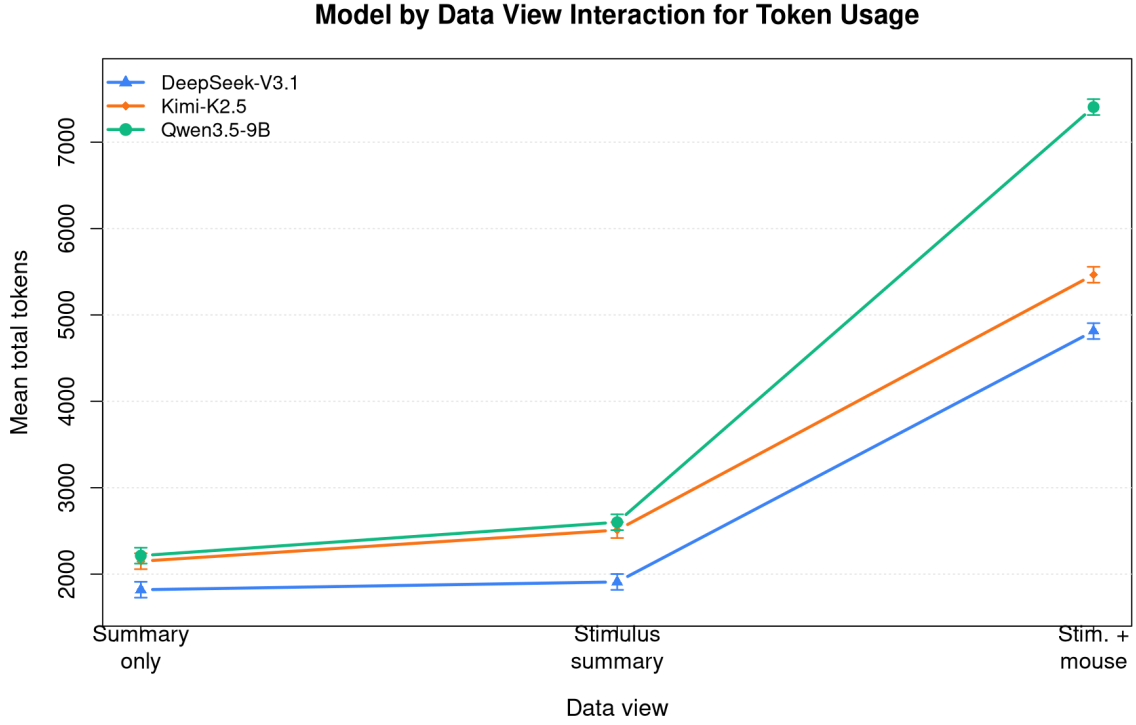


Figure 4: Model-by-data-view interaction for token usage. Error bars show common 95% intervals from the pooled ANOVA error term. Context packaging was the dominant cost lever in the experiment.

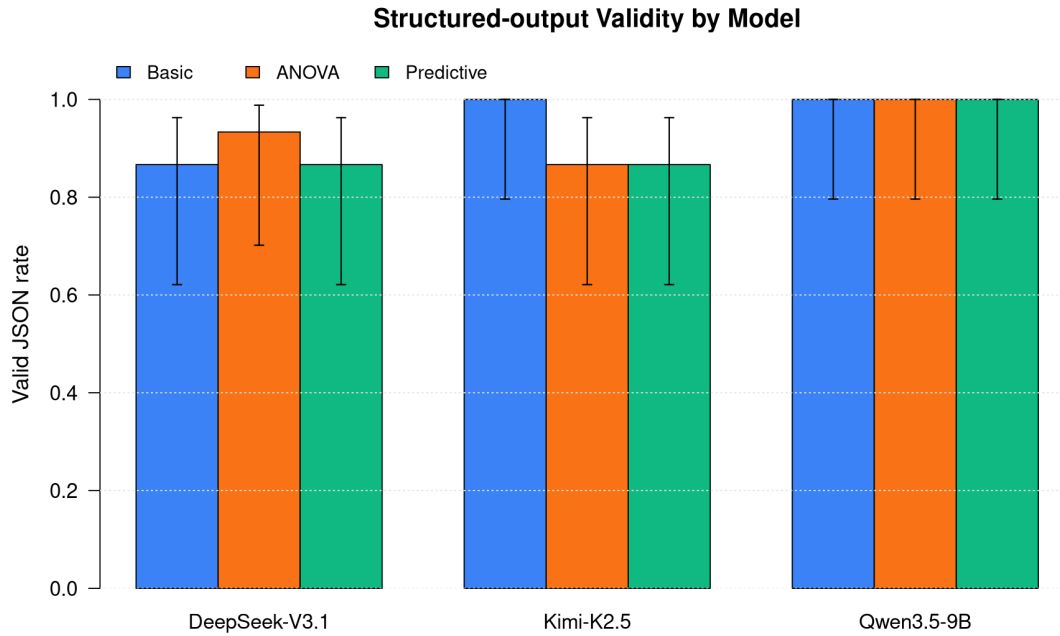


Figure 5: Structured-output validity by model and prompt within the richest data view, under the fixed 1,800-token output cap. Error bars are Wilson 95% intervals with $n = 15$ runs per bar. Reliability was primarily a model effect, not a temperature effect.

Table 6: Additive logistic model summary for valid JSON

Term	Test statistic	p -value
Model	LR $\chi^2 = 41.80$	8.38×10^{-10}
Temperature	—	0.625
Prompt template	—	0.441
Data view	—	0.441

The additive form was used because Qwen3.5-9B achieved perfect validity, which creates separation problems in richer logistic models.

and token cost. DeepSeek-V3.1 was fast and relatively cheap but vulnerable to truncation; Kimi-K2.5 was slower but highly stable; Qwen3.5-9B was most reliable structurally but most token-expensive. Third, *temperature mattered less than expected.* Across the main outcomes, temperature had little independent practical effect compared with model and data-view complexity.

5 Discussion

The key finding is that the computational behavior of LLM-based analysis pipelines is governed primarily by two design choices: which model is used and how much context is embedded in the prompt. This is a useful result for future coursework and small scientific projects because it shows that prompt-context engineering and model selection matter more than temperature sweeps when the objective is efficient, structured statistical output.

Several limitations should be acknowledged. The study measured computational efficiency and output structure, not substantive correctness of the models’ scientific conclusions. A model can return valid JSON quickly while still making questionable inferential claims. In addition, the fixed output cap of 1,800 tokens created a real truncation mechanism, especially for DeepSeek-V3.1. That cap was methodologically useful because it standardized the runtime environment, but it also means that the JSON-validity findings should be interpreted as conditional on a capped-output workflow rather than as an absolute property of each model family.

From a DOE perspective, the design was efficient. A balanced 405-run experiment with three replications cost only about \$1.11, which is exceptionally cheap for a replicated cloud-based experiment. Replication was therefore easy to defend and helped absorb service-level variability that would otherwise have been confounded with treatment effects. The main practical challenge was not budget but the management of structured-output failures and reruns.

Future improvements are clear. The next iteration should add a rubric-based accuracy assessment, possibly combining instructor review with a rules-based checklist for whether the model identified the correct high-level pattern across stimulation paradigms. It would also be valuable to optimize context compression so that richer data views retain their statistical signal without creating such steep token penalties.

6 Code and Data Availability

All project materials are available at <https://github.com/rcalvom/STAT-51400-project> at commit `8b76d1e`. Python was used for experiment execution and Together AI interaction, while R was used to generate report-side summaries and figures. The final balanced results were obtained after rerunning the initial API failures so that all 405 planned runs were present in the completed dataset. Appendix A lists the key repository paths and reproducibility materials.

A Reproducibility Materials

The repository contains the materials needed to reproduce the design and the report:

- raw source dataset: `data.csv`
- derived prompt inputs and summaries: `artifacts/data/`
- experiment configuration: `configs/full_factorial.json`
- randomization seed: 51400
- generated design matrix path: `outputs/design_matrix.csv`
- balanced run-level results: `outputs/results.csv`
- treatment-level summaries: `outputs/results_summary.csv`
- Python execution pipeline: `src/llm_doe/`
- submission-figure script: `analysis/generate_submission_figures.R`
- broader report-artifact script: `analysis/generate_report_artifacts.R`
- R session information: `report/tables/r_session_info.txt`

Within the repository, `data.csv` stores the raw source data. The deterministic prompt-facing summaries live in `artifacts/data/`, and the final run-level DOE outcomes live in `outputs/results.csv`.

References

- [BHH05] George E. P. Box, J. Stuart Hunter, and William G. Hunter. *Statistics for Experimenters: Design, Innovation, and Discovery*. Wiley, 2 edition, 2005.
- [Mon19] Douglas C. Montgomery. *Design and Analysis of Experiments*. Wiley, 10 edition, 2019.
- [Pyt26] Python Software Foundation. *Python 3.14.4 documentation*, 2026. <https://docs.python.org/3/>.
- [R C26] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026.
- [Tog26a] Together AI. Chat and structured outputs. <https://docs.together.ai/docs/chat-overview>, 2026.
- [Tog26b] Together AI. Json mode and structured outputs. <https://docs.together.ai/docs/json-mode>, 2026.
- [Tog26c] Together AI. Serverless models. <https://docs.together.ai/docs/serverless-models>, 2026.